

TITLE: Simulation of Capacitor Discharging

OBJECTIVE:

- I) To simulate the discharging of a capacitor using python
- II) To observe the exponential decrease of capacitor voltage with time.

THEORY:

A Capacitor stores electrical energy in the form of an electric field. When connected across a resistor, it gradually releases its stored energy, causing the capacitor voltage to decrease exponentially over time. This process is known as Capacitor discharging.

The voltage across a discharging capacitor is given by:

$$V(t) = V_0 e^{-t/RC}$$

where,

V_0 = Initial Voltage

R = Resistance

C = Capacitance

RC = Time Constant

In this simulation, the capacitor discharge is approximated using the Euler numerical method.

ALGORITHM:

- 1) Initialize the value of resistance, capacitance, initial voltage, total simulation time, and time step.
- 2) Create a time array.
- 3) Set the initial capacitor voltage.
- 4) Calculate the rate of change of voltage using the capacitor discharge equations.
- 5) Update the voltage using Euler's method.
- 6) Repeat until the simulation ends.
- 7) Plot voltage versus time.

CODE:

simulate the discharging of a capacitor
Exponential decrease

```
import numpy as np
import matplotlib.pyplot as plt
```

```
R = 800
```

```
C = 0.001
```

```
V0 = 10
```

```
t0 = 0
```

```
T = 10
```

```
dt = 0.01
```

```
t = np.arange(t0, T+dt, dt)
```

```
V = np.zeros(len(t))
```

```
V[0] = V0
```

```
for i in range(len(t)-1):
```

```
    dV = -V[i] / (R * C)
```

```
    V[i+1] = V[i] + dV * dt
```

```
plt.figure(figsize=(8,5))
```

```
plt.plot(t, V, color='blue', label='Capacitor Discharging')
```

```
plt.xlabel("Time (s)")
```

```
plt.ylabel("Voltage (V)")
```

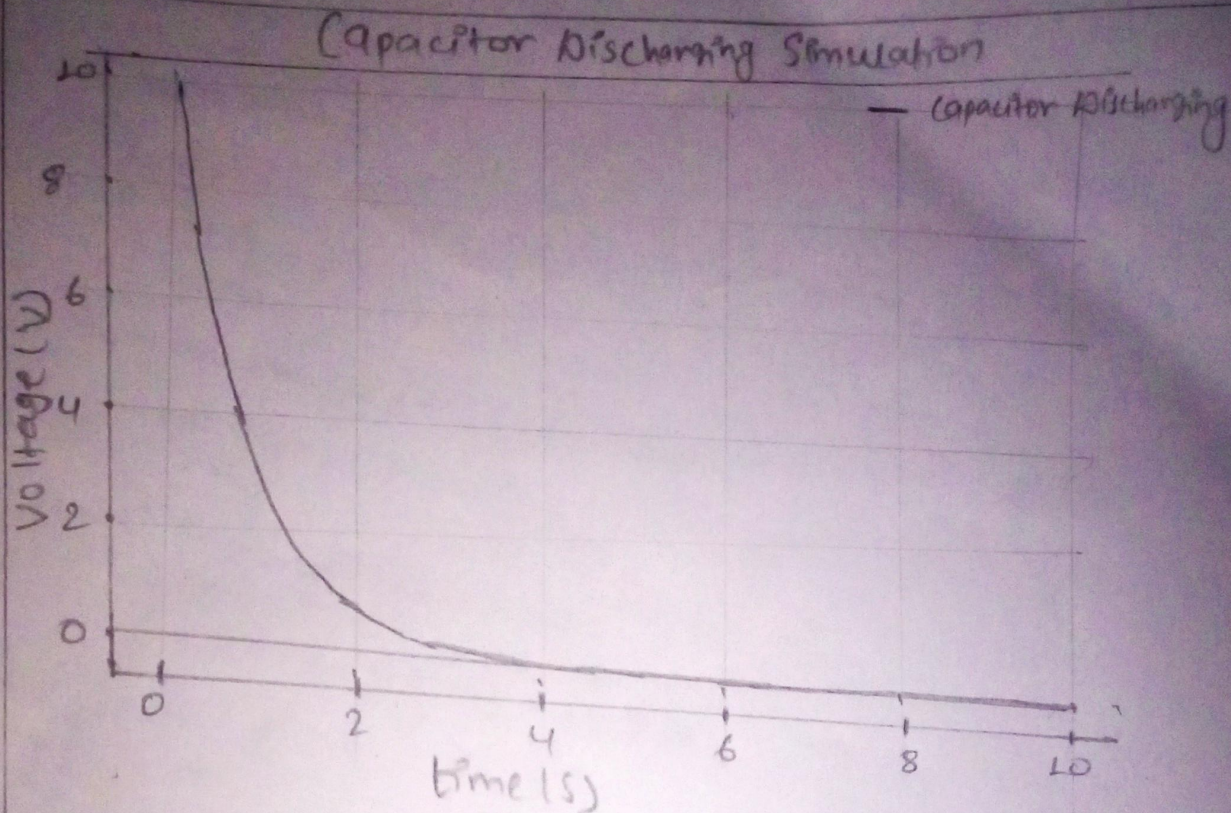
```
plt.title("Capacitor Discharging Simulation")
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.show()
```

Output:



RESULT:

The Capacitor discharging process was successfully simulated using Python. The Voltage decreased exponentially from the initial value of 10V toward 0V, which agrees with the theoretical behavior of an RC discharging circuit.

CONCLUSION:

The simulation successfully demonstrated the discharging behavior of a Capacitor using the Euler numerical method.

TITLE: Simulation of continuous System (Spring-Mass-Damper System)

OBJECTIVE:-

- 1) To simulate the behavior of a spring-mass-damper system using python.
- 2) To study the effect of different damping ratios on the displacement of the system.

THEORY:

A Spring-mass-damper system is a continuous dynamic system consisting of a mass attached to a spring and a damper. When an external force is applied, the system oscillates. The damping ratio determines how quickly these oscillations decrease. Depending on the damping ratio, the system may be underdamped, critically damped, overdamped, or unstable.

The motion of the system is governed by a second-order differential equation. In this simulation, the RK4 order numerical method is used to solve the equation and observe the displacement over time.

ALGORITHM:

1. Initialize system parameters such as force, angular frequency, simulation time, and time step.
2. Set the initial displacement and velocity to zero.
3. Define the differential equations of the spring-mass-damper system.
4. Select different damping ratios.
5. Solve the differential equations using the RK4 method.
6. Store the displacement values at each time step.
7. Plot displacement versus time for each damping ratio.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
F=5
```

```
w=0.5
```

```
t0=0
```

```
T=50
```

```
dt=0.01
```

```
N=int((T-t0)/dt)
```

```
t=np.arange(0, N)*dt
```

```
def du(u, z):
```

```
    return np.array([u[1], (w**2)*(F-u[0]) - 2*z*w*u[1]])
```

```
u=np.zeros((N,2))
```

```
u[0,:]=[0,0]
```

```
plt.figure(figsize=(10,6))
```

```
damping_ratios = [-0.2, 0.01, 1, 2]
```

```
line_styles = ['-', '--', ':', '-.']
```

```
for k, z in enumerate(damping_ratios):
```

```
    u[:,0]=0
```

```
    u[0,:]=[0,0]
```

```
    for i in range(N-1):
```

```
        m1=du(u[i:], z)
```

```
        m2=du(u[i:] + m1/2, z)
```

```
        m3=du(u[i:] + m2/2, z)
```

```
        m4=du(u[i:] + m3, z)
```

```
        u[i+1,:]=u[i:] + dt*(m1 + 2*m2 + 2*m3 + m4)/6
```

```
plt.plot(t, u[:,0], label=f'z={z}', linestyle=line_styles[k])
```

```
plt.xlabel("Time (s)")
```

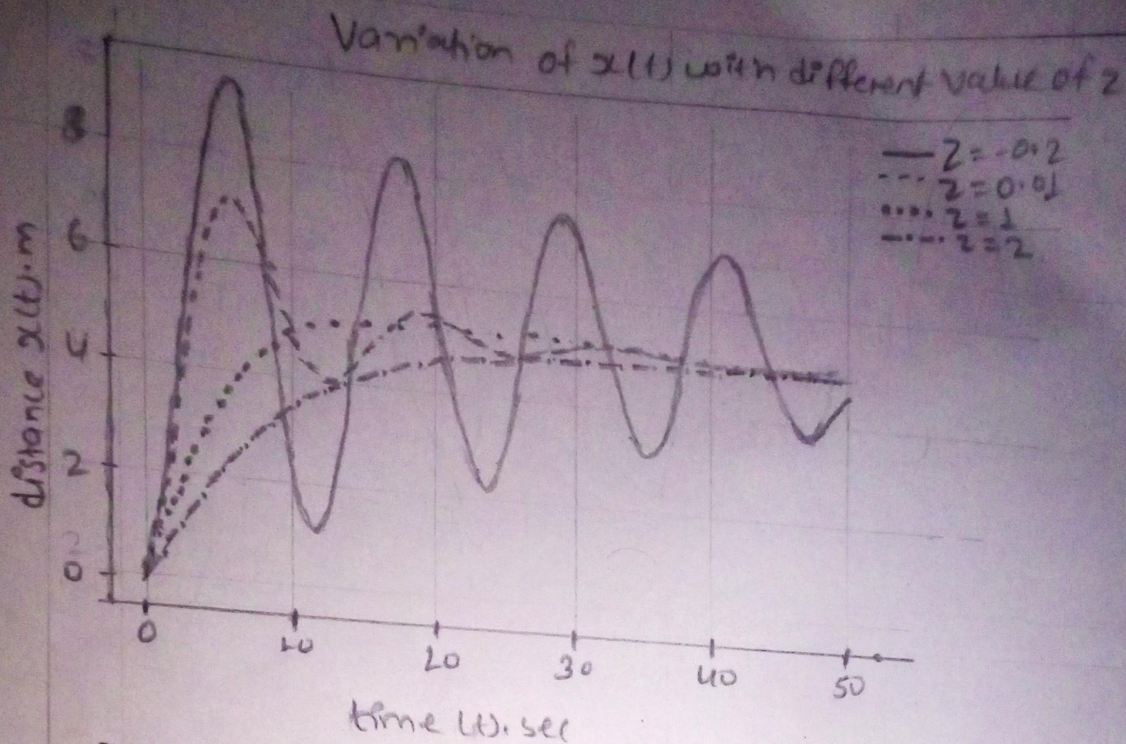
```
plt.ylabel("Displacement x(t) (m)")
```

```
plt.title("Spring-Mass-Damper System")
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.show()
```



- $z = -0.2$: Oscillations increase initially, indicating an unstable system due to negative damping.
- $z = 0.01$: The system is lightly damped and oscillates for a longer time before settling.
- $z = 1$: The system reaches equilibrium quickly with very little oscillation (critical damping behavior).
- $z = 2$: The system is overdamped and slowly reaches the equilibrium without oscillating.

RESULT:

The simulation successfully demonstrates the effect of different damping ratios on the spring mass damper system. As the damping ratio increases, oscillations decrease and the system reaches the equilibrium position more smoothly.

CONCLUSION:

The spring-mass-damper system was successfully simulated using Python. The RK4 numerical method accurately modeled the system response.

TITLE: Find the Value of π Using Monte Carlo Simulation

OBJECTIVE:

- i) To estimate the value of π using the Monte Carlo Simulation method.
- ii) To generate random points and determine whether they lie inside or outside a quarter circle.

THEORY:

Monte Carlo Simulation is a statistical technique that uses random sampling to estimate numerical values. To estimate π , random points are generated inside a unit square (1×1). A quarter circle of radius 1 is inscribed inside the square. The ratio of the number of points inside the quarter circle to the total number of points is approximately equal to $\pi/4$.

The value of π is calculated using:

$$\pi \approx 4 \times \frac{\text{Number of points inside the quarter circle}}{\text{Total no. of points}}$$

As the no. of random points increases, the estimated value becomes closer to the actual value of π .

ALGORITHM:

1. Generate N random points inside a unit square.
2. Calculate the distance of each point from the origin.
3. Check whether the point lies inside the quarter circle.
4. Count the no. of points inside the quarter circle.
5. Estimate the value of π using the Monte Carlo formula.
6. Display the estimated value and plot the points.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
N = 1000000
```

```
x = np.random.rand(N)
```

```
y = np.random.rand(N)
```

```

inside = x**2 + y**2 <= 1
pi = 4 * np.sum(inside) / N
print("Estimated value of pi:", pi)
print("Actual value of pi:", np.pi)
plt.figure(figsize=(8,8))
plt.scatter(X[inside], Y[inside], color='r', s=3, label='inside circle')
plt.scatter(X[~inside], Y[~inside], color='b', label s=3, label='Outside Circle')
plt.xlabel("x")
plt.ylabel("y")
plt.title(f"Monte Carlo Estimation of pi in Estimated  $\pi = \frac{2 \text{ pi} \times 4 \text{ f } 2}{2}$ ")
plt.legend()
plt.axis("equal")
plt.grid(True)
plt.show()

```

Output:

Estimated value of pi: 3.1418

Actual value of pi: 3.141592653

RESULT:

The Monte Carlo simulation successfully estimated the Value of π as 3.1418, which is very close to the actual Value of 3.1416. This demonstrates that the Monte Carlo method provides an accurate approximation when a large number of random samples are used.

CONCLUSION:

The Value of π was successfully estimated using the Monte Carlo simulation method.